SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

KudoScan is an Autonomous Phishing Triage Copilot built for UMHackathon 2026. It addresses the growing threat of phishing attacks targeting corporate employees by providing an intelligent, fully automated scam detection system accessible via Telegram.

1.1. Purpose:

This document outlines the technical architecture and design decisions behind KudoScan. Key elements covered include system architecture, data flows, the GLM reasoning pipeline, VirusTotal threat intelligence integration, SQLite-based SIEM logging, and the Streamlit web dashboard.

The key element of system that has been covered are as followed:

1. **Architecture**:
    a. KudoScan is structured as a multi-tool pipeline deployed on a single server. The Telegram bot acts as the user-facing interface, which feeds into a backend orchestration layer that coordinates Regex extraction, OCR, VirusTotal, and GLM. Regex serves as the first-pass extraction layer for unstructured text inputs, while OCR.space handles image-based submissions separately.
2. **Data Flows:**
    a. A suspicious message or image submitted by a user flows from Telegram into the bot backend, where Regex automatically hunts for and extracts URLs and IP addresses from unstructured text, or OCR.space extracts them from image screenshots. VirusTotal then scans the extracted URL for threats, and ILMU-GLM-5.1 reasons over the combined results, which returns a strict JSON response with threat level, confidence score, and reasoning to produce a structured threat report. The outcome is logged to SQLite and surfaced on the Streamlit dashboard.
3. **Model Process:**
    a. The end-to-end workflow covers six automated steps:
    (1) Regex-based URL and IP extraction from unstructured text, or Vision OCR extraction from image screenshots
    (2) VirusTotal API-driven threat intelligence scanning
    (3) ILMU-GLM-5.1 reasoning with strict JSON output (threat level, confidence score, reasoning)
    (4) Automated Risk Matrix evaluation that dynamically changes pipeline behaviour based on threat level
    (5) OpSec URL defanging followed by SQLite SIEM logging and real-time dashboard update. For High/Critical threats, a Human-in-the-Loop (HITL) authorization step halts the pipeline pending SOC admin approval
4. **Role of Reference:**
    a. This document serves as a technical reference for the development team, hackathon judges, and evaluators to understand the high-level architecture, data flow, and design rationale behind KudoScan.

© UMHackathon 2026

1.2. Background:

Phishing attacks remain one of the most persistent cybersecurity threats in corporate environments. IT and SOC teams are overwhelmed by the volume of suspicious emails and messages reported by employees, with manual triage taking up to 20 minutes per incident. KudoScan was conceived to fully automate this Level 1 SOC workflow.

1. Previous Version: Prior to KudoScan, phishing triage relied on manual processes: an IT worker would manually transcribe a suspicious URL from a screenshot, query threat databases individually, and write up an incident report. There was no centralized logging or adaptive caching of previously seen threats.
2. Changes in Major Architectural Components: The shift from manual triage to KudoScan represents a full architectural replacement rather than an incremental update. What was previously a human-driven, multi-step process has been replaced by an automated sequential pipeline. The human role has moved from executor to approver, with SOC staff only intervening at the final escalation stage for High-severity threats.
3. New capabilities that are only introduced in KudoScan Bot: A Regex engine (threat_engine.py) that automatically extracts URLs and IP addresses from messy unstructured text; a Vision OCR engine that extracts URLs directly from image screenshots, eliminating the need for manual transcription; ILMU-GLM-5.1 operating under a "Level 3 SOC Analyst" persona with forced JSON output for programmatic downstream processing; an Automated Risk Matrix that dynamically changes pipeline behaviour based on threat grade; a Human-in-the-Loop (HITL) authorization gate that halts the workflow for High/Critical threats and requires explicit admin approval via Telegram Inline Keyboard; a Zero-Trust OpSec URL defanging shield that neutralises malicious links before display; a Global Cache with adaptive memory that auto-escalates previously confirmed threats; and a real-time Streamlit dashboard for C-suite incident visibility.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Users	Forwards suspicious messages, images, or URLs to the KudoScan Telegram bot and receives an instant threat verdict	Fast, accurate phishing detection with clear threat verdicts and zero manual effort required.
SOC / IT Admin	Reviews the Streamlit dashboard for logged	Clear incident logs, accurate threat grades, and an intuitive

© UMHackathon 2026

	incidents and approves high-severity block actions	dashboard for monitoring submissions
C-Suite / Management	Views the real-time Streamlit dashboard for organizational threat visibility	High-level summary of phishing attempts, threat trends, and auto-blocked URLs across the organization.
Development Team	Build and maintain the KudoScan pipeline, bot, and dashboard	Clear API usage contracts, codebase is modular and well-documented architecture
QA Team	Validate threat detection accuracy, pipeline reliability, and dashboard correctness	Defined scope, the request and response formats, and coverage of edge cases

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Telegram Bot + Streamlit Web Dashboard
Architecture	Single-server pipeline with modular tool integration

KudoScan is structured as a sequential orchestration pipeline. Employees interact with the system via Telegram, submitting suspicious messages, images, or URLs. The backend processes submissions through five stages: OCR extraction (OCR.space), threat scanning (VirusTotal API), GLM reasoning (Z.ai GLM), URL redaction, and SQLite logging. A Streamlit dashboard provides C-suite visibility into all logged incidents in real time.

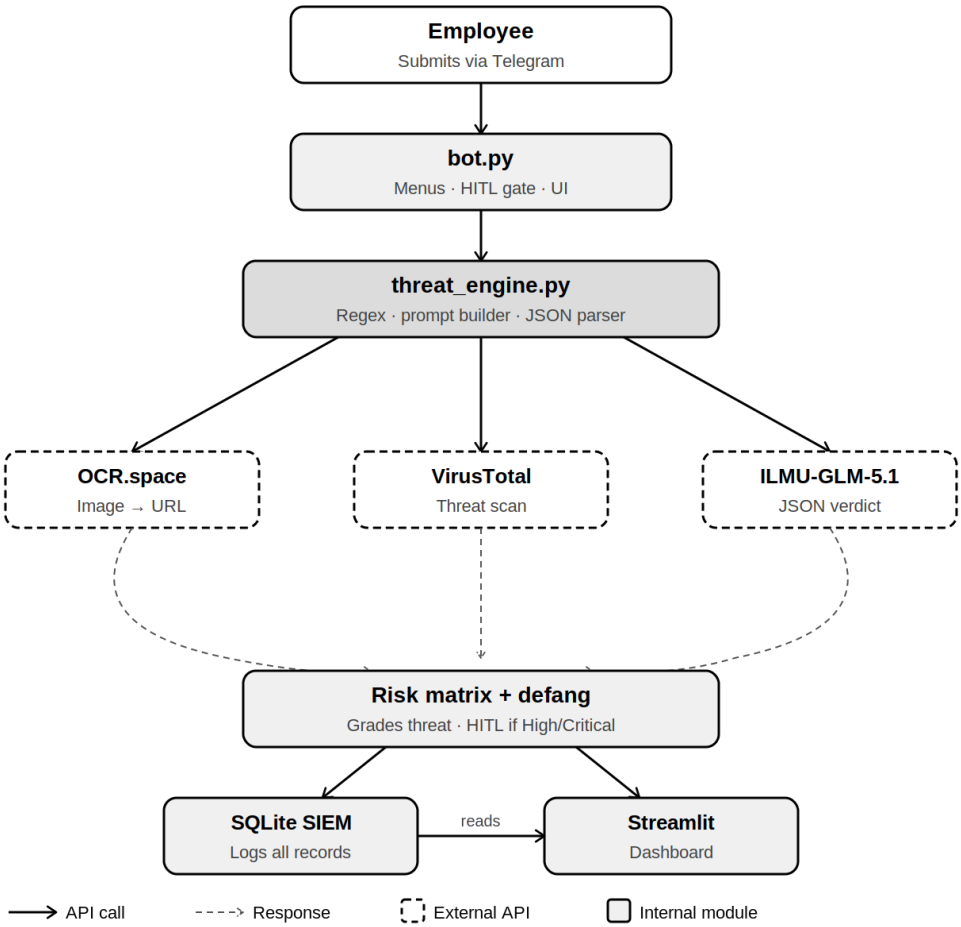
2.1.2 LLM as Service Layer

In KudoScan, ILMU-GLM-5.1 is deployed as a structured reasoning service layer with a hardcoded “Level 3 SOC Analyst” persona, giving it a defined role and decision-making frame rather than operating as a generic AI module. Prompts are constructed

© UMHackathon 2026

programmatically by the backend orchestrator in threat_engine.py, injecting two variable inputs into a fixed template: the raw URL or text extracted via Regex or OCR, and the structured JSON response returned by VirusTotal containing vendor detection counts and threat categories. Critically, GLM is instructed to return its output strictly as JSON that contains threat level, confidence score, and reasoning so that downstream Python code can programmatically read and act on the AI's decisions rather than parsing free-form text. This composed prompt is sent to the GLM API via a POST request with a global fallback timeout of 60 seconds to ensure predictable latency and prevent hung processes. Input length is controlled at the injection stage, where OCR output and VirusTotal JSON are truncated to defined character limits before being inserted into the prompt, preventing token overflow and ensuring cost consistency across sessions.

2.1.3 Dependency Diagram



a. Prompts are constructed programmatically in threat_engine.py using a fixed template that injects two key inputs: the extracted URL or text (from Regex or OCR) and the structured VirusTotal JSON results. The GLM is assigned a predefined "Level 3 SOC Analyst" persona to standardize its reasoning, and the final prompt is sent to the

GLM API via a POST request with a global timeout to ensure predictable performance and prevent stalled processes.

b. The context window consists of the extracted user input (URL, IP, or text), OCR output for image-based submissions, and VirusTotal scan results in JSON format, including detection counts and threat categories. These inputs are combined with fixed instructions that enforce a strict JSON output format containing threat level, confidence score, and reasoning.

c. The GLM returns a strictly structured JSON response, which is directly parsed by threat_engine.py without the need for manual text interpretation. The parsed values are then passed to bot.py, where the Risk Matrix determines the next action, including possible HITL escalation for high-risk threats, before sending a response to the user, logging the incident in SQLite, and updating the Streamlit dashboard.

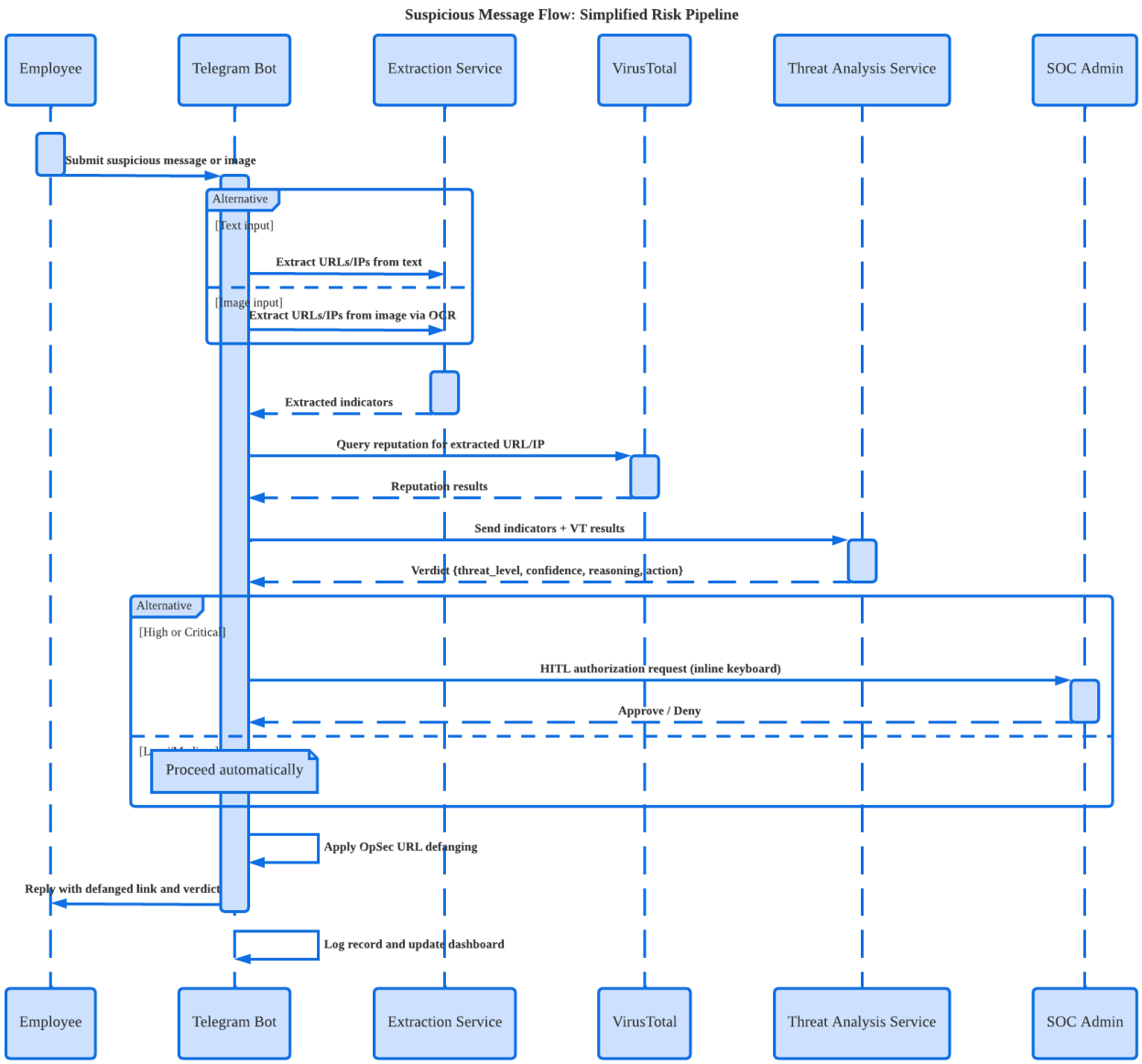
d. Token limits are enforced during the prompt construction stage, where OCR output and VirusTotal JSON are truncated to predefined character limits before being inserted into the prompt. This ensures that inputs remain within token constraints, avoids overflow, and maintains consistent cost and latency without requiring dynamic input chunking.

e. All major API calls: Telegram bot (bot.py) calls OCR.space API for image text extraction; threat_engine.py applies Regex for text-based URL/IP extraction; threat_engine.py calls VirusTotal API v3 with base64-encoded URL; threat_engine.py calls ILMU-GLM-5.1 API with a structured prompt containing Regex/OCR output + VirusTotal JSON; bot.py writes threat record to SQLite; Streamlit reads from SQLite for dashboard rendering.

f. Telegram (frontend) communicates with bot.py, which delegates to threat_engine.py for all intelligence work. threat_engine.py calls OCR.space or applies Regex, then calls VirusTotal, then constructs a fixed-template prompt injected with extracted URL and VirusTotal JSON before calling ILMU-GLM-5.1. The strictly formatted JSON response is parsed for threat level, confidence score, and reasoning. bot.py then applies OpSec defanging, evaluates the Automated Risk Matrix, triggers HITL if required, writes to SQLite, and updates the Streamlit dashboard.

2.2.2. Sequence Diagram

© UMHackathon 2026



a. Phishing Triage Flow:

(1) Employee submits suspicious message or image to Telegram bot via Inline Keyboard menu or direct forward.

(2) Bot applies Regex to extract URLs/IPs from text input, or sends image to OCR.space for URL extraction.

(3) Bot queries VirusTotal API v3 with the extracted URL (base64-encoded).

(4) threat_engine.py constructs a structured prompt with Regex/OCR output + VirusTotal JSON and sends to ILMU-GLM-5.1.

(5) ILMU-GLM-5.1 returns a strict JSON response containing threat level, confidence score, and reasoning.

(6) Automated Risk Matrix evaluates the threat grade and determines pipeline action.

© UMHackathon 2026

(7) For High/Critical threats, bot halts and presents HITL authorization request to SOC admin via Telegram Inline Keyboard.
(8) Bot applies OpSec URL defanging, replies to user with defanged link and verdict, logs full record to SQLite, and Streamlit dashboard updates with new entry.

2.2. Technological Stack

2.2.1. Frontend – Telegram Bot API (user interface for employees) + Streamlit (web dashboard)

Telegram serves as the employee-facing submission interface, leveraging Inline Keyboards for guided menus and live message rewriting to display a transparent terminal-style analysis sequence ([INIT], [EXTRACTED], [DATA ACQUIRED]), while Streamlit provides a lightweight, Python-native web dashboard for SOC admins and C-suite stakeholders to monitor logged incidents in real time.

2.2.2. Backend – Python (orchestration logic, API integrations, threat grading pipeline)

Python serves as the backbone of KudoScan's orchestration layer, with threat_engine.py handling Regex extraction, VirusTotal API calls, ILMU-GLM-5.1 prompt construction, and JSON response parsing, while bot.py manages the Telegram interface, Inline Keyboard menus, HITL authorization flow, and live terminal animation.

2.2.3. Database – SQLite (lightweight SIEM database for logging all threat incidents)

SQLite provides a serverless, zero-configuration relational database for persisting all triage records, including extracted URLs, VirusTotal results, GLM verdicts, and cache status, making it well-suited for a self-contained hackathon deployment.

2.2.4. External APIs – OCR.space (Vision/OCR), VirusTotal API v3 (Threat Intelligence), Z.ai GLM API (GLM Reasoning Engine)

OCR.space handles image-to-text extraction for screenshot-based submissions, VirusTotal provides multi-vendor URL threat intelligence, and Z.ai GLM serves as the reasoning engine that synthesises all signals into a structured, human-readable threat verdict.

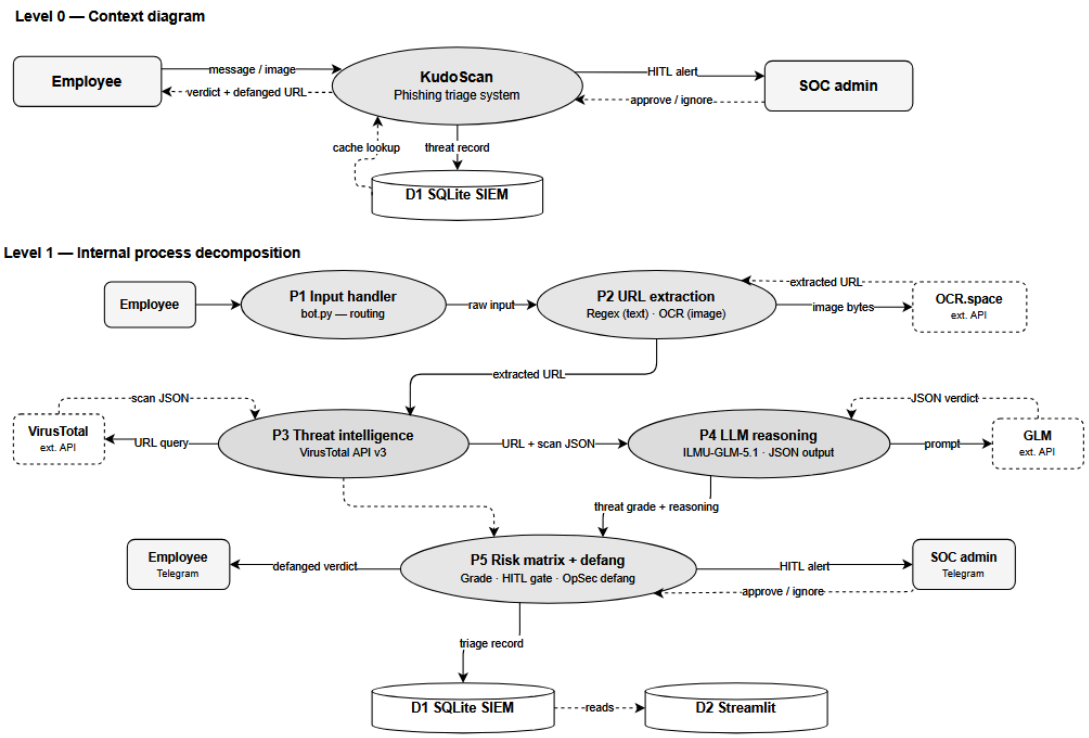
2.3. Key Data Flows

2.3.1. When an employee forwards a suspicious message or image to the KudoScan Telegram bot, the input is routed through bot.py to threat_engine.py, where Regex automatically extracts URLs and IP addresses from text-based inputs, or OCR.space is called to extract URLs from image screenshots. The extracted URL is then passed to the VirusTotal API v3 for multi-vendor threat scanning, and the resulting scan JSON is combined with the extracted URL into a structured prompt that is sent to ILMU-GLM-5.1, which returns a strict JSON

© UMHackathon 2026

response containing a threat grade, confidence score, and reasoning. The Risk Matrix in bot.py reads this JSON output and dynamically determines the next action — replying directly to the employee with a defanged verdict for Low and Medium threats, or halting the pipeline and issuing a Human-in-the-Loop authorization request to the SOC admin via Telegram Inline Keyboard for High and Critical threats. Regardless of outcome, every triage event is written as a structured record to the SQLite SIEM database, which the Streamlit dashboard reads in real time to provide C-suite stakeholders with live visibility into all logged incidents.

2.3.1.1. Data Flow Diagram (DFD)



.

2.3.2. Normalized Database Schema

ThreatLog

Column	Type	Notes
id	INTEGER	**PK, autoincrement**
timestamp	TEXT	NOT NULL
submitted_by	TEXT	Telegram user ID
raw_input	TEXT	Original submission
extracted_url	TEXT	**FK → GlobalCache**
virustotal_result	TEXT	Raw JSON
glm_grade	TEXT	Low / Medium / High / Critical
glm_confidence	TEXT	0.0 – 1.0
glm_reasoning	TEXT	GLM explanation
redacted_url	TEXT	Defanged link
cache_status	TEXT	NEW / CACHE HIT
hitl_decision	TEXT	APPROVED / IGNORED / NULL
pipeline_status	TEXT	COMPLETE / ERROR / DEGRADED

references (many-to-one)

GlobalCache

Column	Type	Notes
id	INTEGER	**PK, autoincrement**
url_hash	TEXT	**UNIQUE, NOT NULL**
extracted_url	TEXT	Original URL
glm_grade	TEXT	Cached verdict
glm_reasoning	TEXT	Cached reasoning
virustotal_result	TEXT	Cached VT JSON
hit_count	INTEGER	Repeat submissions
first_seen	TEXT	First submission timestamp
last_seen	TEXT	Most recent submission
auto_block_status	TEXT	NONE / AUTO_BLOCKED

© UMHackathon 2026

a. The SQLite schema contains a single primary table named incidents. The core columns logged for every event include timestamp, target_type, target_value (the defanged URL or IP), threat_level, and recommended_action. Instead of storing redundant cache statuses, the Global Cache's adaptive memory feature operates dynamically by querying the target_value column. If a match is found, the system instantly retrieves the historical threat_level and recommended_action without needing to re-trigger the external threat intelligence APIs.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects that focuses on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

The following table outlines the 3 core MVP features to be demonstrated at UMHackathon 2026.

No.	Features	Description
1	Phishing URL/Message Detection	Employee submits a suspicious message, URL, or image via Telegram; bot returns a threat verdict (Low/Medium/High) with reasoning.
2	Image OCR Extraction	When a suspicious image is submitted, OCR.space extracts embedded URLs automatically, enabling analysis even when users cannot type out the link manually.
3	SIEM Logging & Dashboard	Every triage event is logged to SQLite and surfaced on the Streamlit dashboard for SOC and management review.

3.2. Non-Functional Requirements (NFRs)

System qualities that are not features but are crucial for reliability and improve the system, as outlined below.

Quality	Requirements	Implementation
Scalability	KudoScan must handle concurrent threat submissions from multiple employees without queuing failures.	Telegram bot uses async handlers; pipeline steps are non-blocking to support concurrent users.
Reliability	Every submission must produce either a completed threat record or a graceful error	SQLite writes are transactional; if the GLM or VirusTotal call

© UMHackathon 2026

	response. No orphaned partial records in SQLite.	fails, the record is marked with an error status rather than being dropped.
Maintainability	Each pipeline module (OCR, VirusTotal, GLM, DB) is encapsulated in its own Python function for independent testing and replacement.	Modules can be swapped independently (e.g. replacing OCR.space with a local model) without touching the orchestration logic.
Token Latency	GLM API calls must return a response within 5s under normal load; VirusTotal scans within 3s.	Timeout of 60s enforced on all external API calls. Bot sends a "Analyzing..." message immediately to indicate processing is underway.
Cost Efficiency	Average token cost per triage session must remain within the Z.ai GLM free-tier limits for the hackathon duration.	Prompts are fixed-length templates with variable slots for OCR output and VirusTotal JSON. Repeated URL submissions are served from the Global Cache, bypassing GLM and VirusTotal calls entirely.

3.3. Out of Scope / Future Enhancements

The following features are planned for the grand vision of KudoScan but are out of scope for UMHackathon 2026.

a. Multi-language phishing detection — KudoScan currently reasons over English-language content. A future enhancement would extend GLM prompt handling to detect phishing attempts in other languages (e.g. Malay, Mandarin), which is especially relevant given the Malaysian corporate context.

b. Email/SMS gateway integration — Rather than requiring employees to manually forward suspicious content to Telegram, a future version could integrate directly with corporate email (e.g. via Gmail or Outlook API) and SMS gateways to automatically intercept and triage suspicious messages without any user action.

c. Automated URL blocking via network firewall — Currently, KudoScan flags High-severity threats and requests a human to approve the block. A future enhancement would integrate with corporate network firewall or DNS filtering tools (e.g. Cloudflare Gateway) to automatically push confirmed malicious URLs to a blocklist, completing the loop from detection to enforcement without human intervention.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: For hackathon purposes, KudoScan is deployed to a single test environment. Post-hackathon, a phased rollout would begin with a pilot group of 10 employees, monitoring false positive rates before expanding to the full organization. A/B testing would compare GLM prompt variants to optimize threat grading accuracy. The deployment strategy was released to 5% of real users initially, monitoring and then gradually increased until 100% if no issues are detected. After it passes, A/B testing is conducted to determine which variants of two strategies perform better.

4.1.2. **Emergency Rollback (ER) & Golden Release**: If the GLM or VirusTotal API becomes unavailable, KudoScan falls back to URL-pattern heuristics and cached results. The last stable bot version is preserved as the Golden Release and can be redeployed within minutes via a single script.

4.1.3. Priority Matrix:

Priority Level	Trigger Condition	Action Taken
P1 - Critical	VirusTotal API is unreachable or returns errors for 3 or more consecutive requests	Immediately switch to GLM-only heuristic mode; surface a system alert on the Streamlit dashboard; notify SOC admin; log all affected submissions with a "UNVERIFIED" status flag
	GLM API is unreachable or times out for 3 or more consecutive requests	Halt triage pipeline; return a graceful error message to the Telegram user; trigger emergency rollback to last Golden Release; alert SOC admin
P2 - High	OCR.space API is unavailable	Disable image-based submission handling; prompt users to submit URLs as plain text instead; continue pipeline normally for text-based inputs
	VirusTotal rate limit exceeded (free tier cap hit)	Queue incoming requests and process with a delay; notify SOC admin of rate limit status; switch to cached results for previously seen URLs

© UMHackathon 2026

P3- Medium	SQLite write failure on incident logging	Buffer the failed log entry in memory and retry up to 3 times; if retries fail, output the incident report to a local fallback log file to prevent data loss
	Telegram Bot API becomes unresponsive	Bot attempts reconnection with exponential backoff; SOC admin is alerted if downtime exceeds 5 minutes
P4 - Low	Streamlit dashboard fails to render or refresh	Core triage pipeline remains unaffected; SOC admin is notified to restart the dashboard service manually; incident data remains intact in SQLite

4.2. Monitoring:

At the API layer, the bot performs a lightweight heartbeat check against the VirusTotal, GLM, and OCR.space endpoints at a regular interval; if any endpoint fails to respond within the defined timeout threshold, the bot immediately logs the failure, updates the Streamlit dashboard with a system health warning banner, and notifies the SOC admin via a Telegram alert message to a designated admin chat ID. At the pipeline layer, every triage job is wrapped in a try-catch block that tracks success and failure counts per session; if the failure rate for any pipeline stage exceeds 20% within a rolling window, the system automatically flags that stage as degraded and triggers the corresponding priority level from the Priority Matrix. At the database layer, a write-integrity check is performed after every SQLite INSERT. If a write acknowledgement is not received, the entry is buffered in memory and a P3 alert is raised. The Streamlit dashboard serves as the central health surface, displaying a live status indicator for each external dependency (VirusTotal, GLM, OCR.space) alongside triage throughput metrics, so that SOC admins and C-suite stakeholders have immediate visibility into any degradation without needing to inspect logs directly.

4.3. Assumptions:

Key assumptions for KudoScan:

a. Employees have access to Telegram on their devices.

b. Submitted content is in a supported format (plain text, a direct URL, or a standard image format (JPEG/PNG screenshot)).

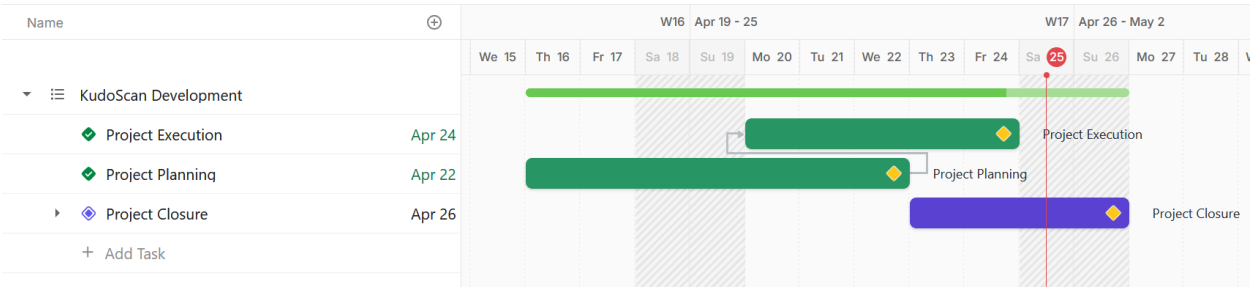
c. Z.ai GLM API and VirusTotal API remain available during the hackathon demo. VirusTotal's free-tier API quota (4 lookups per minute, 500 per day) is assumed to be sufficient for the hackathon demonstration volume. The system has not been architected for high-throughput production traffic beyond this threshold.

4.4. External Dependencies:

Tools	Purpose	Risks
Z.ai GLM API	The core GLM reasoning engine that will handle the prompt inference, multi-step reasoning and generating structured output	High — if GLM API rate limits are hit, triage will stall. Mitigation: implement graceful degradation using heuristic URL analysis and cached results to maintain basic functionality.
OCR.space API	Vision/OCR extraction of URLs from suspicious image screenshots submitted by employees	Medium — if OCR.space API is unavilable, image-based submissions will fail. Mitigation: fallback to manual URL text inpaut.
VirusTotal API	Provides multi-vendor threat intelligence by scanning submitted URLs against 70+ security engines and malware databases	High — if VirusTotal goes offline or hits rate limits, threat verdicts will lack external validation, significantly reducing detection accuracy. Mitigation is to fall back to GLM-only heuristic analysis based on URL structure and patterns, with a warning surfaced to the user that results are unverified by external threat intelligence.

5. Project Management & Team Contributions

5.1. Project Timeline:



© UMHackathon 2026

3 Tasks	Assignee	Status	Priority	Start date	Due date	Project ...	Project Progress
Project Planning 3		COMP...		4/16/26	3 days ago	Planni...	100%
Project Planning and Architecture Design		COMP...		4/16/26	4/18/26	Planni...	100%
API Key Setup		COMP...		5 days ago	3 days ago	Planni...	100%
Product Requirement Document (PRD) ...		COMP...		5 days ago	3 days ago	Planni...	100%
Project Execution 9		COMP...		5 days ago	Yesterday	Execut...	100%
Telegram Bot Scaffolding		COMP...		5 days ago	Yesterday	Execut...	100%
OCR.space Integration		COMP...		5 days ago	Yesterday	Execut...	100%
VirusTotal API Integration		COMP...		4 days ago	2 days ago	Execut...	100%
GLM Prompt Engineering		COMP...		5 days ago	Yesterday	Execut...	100%
SQLite SIEM Logging Implementation		COMP...		4 days ago	Yesterday	Execut...	100%
System Analysis Document (SAD) Prepa...		COMP...		3 days ago	Yesterday	Execut...	100%
Adaptive Workflow Implementation		COMP...		3 days ago	Yesterday	Execut...	100%
Streamlit Dashboard Development		COMP...		5 days ago	Yesterday	Execut...	100%
QA Testing Documentation Preparation		COMP...		2 days ago	Yesterday	Execut...	100%
Project Closure 5		IN PRO...		2 days ago	Tomorrow	Closure	40%
Testing & Validation		COMP...		2 days ago	Today	Closure	100%
Bug Fixes		COMP...		2 days ago	Today	Closure	100%
Slide Deck Preparation		IN PRO...		Yesterday	Tomorrow	Closure	100%
Protoype Demo Video Preparation		IN PRO...		Today	Tomorrow	Closure	100%
Documentation Finalization		IN PRO...		Today	Tomorrow	Closure	100%

5.2. Team Members Role:

Name	Role	Contribution
Balqis Nabilah binti Azly	Project Manager	Led project planning, coordinated team tasks, managed risks and dependencies, and ensured timely delivery of all components aligned with the defined architecture and MVP scope.
Izzatul Najwa binti Che Zaki	Dashboard Engineer	Developed the Streamlit dashboard to visualize real-time incident data, integrate with SQLite, and display system health and threat insights for stakeholders.
Fatin Nabihah binti Massudi	QA Lead	Ensured system accuracy and reliability by testing the end-to-end pipeline, validating outputs, handling edge cases, and verifying correct logging and dashboard data.
Daphne binti Douglas	Backend and AI Engineer	Engineered the ILMU-GLM-5.1 reasoning pipeline and two-stage prompt design and Architected the central Python pipeline with Telegram Bot integration.

© UMHackathon 2026

5.3. Recommendations:

a. Replace SQLite with PostgreSQL for multi-user concurrency — PostgreSQL supports true multi-user concurrent writes with row-level locking, making it far more suitable for a production corporate deployment. It also provides better support for complex queries and indexing, which will be valuable as the ThreatLog table scales into the hundreds of thousands of records.

b. Migrate the Global Cache to Redis for distributed, TTL-based caching — The current Global Cache is stored in SQLite, meaning it is tied to a single server instance and has no automatic expiry mechanism for stale threat entries. Redis would allow the cache to be shared across multiple bot instances in a scaled deployment, and its native TTL support ensures that cached threat verdicts automatically expire after a defined period, which is important because a URL that was safe yesterday may be flagged as malicious today.

c. Add a dedicated URL sandbox (e.g. URLScan.io API) for deeper analysis of live phishing pages — VirusTotal provides reputation-based scanning against known threat databases, but it does not actively load and inspect a live page's behaviour in real time. Integrating a sandboxing service like URLScan.io would allow KudoScan to detonate suspicious URLs in a safe, isolated environment and capture screenshots, redirect chains, and JavaScript behaviour. This is particularly valuable for zero-day phishing pages that have not yet been indexed by VirusTotal's vendor network.

d. Implement role-based access control on the Streamlit dashboard for production use — The current Streamlit dashboard is unauthenticated and accessible to anyone with the URL, which is acceptable for a hackathon demo but poses a serious security risk in a production environment where sensitive incident data is being logged. Implementing RBAC would allow different access tiers. For example, C-suite stakeholders seeing only aggregate summaries while SOC admins have full access to raw logs and system health indicators.

© UMHackathon 2026